챕터 6. 전화, 알림, 인터폰. Polling · SSE · WebSocket

이번 챕터가 끝나면

- 실시간 통신 세 가지 방식(**Polling · SSE · WebSocket**)을 같은 시나리오(채팅)에 붙여 보겠습니다
- 2초마다 전화 거는 **Polling**을 `setInterval`로 만들어 보고, 빈 응답이 쌓이는 모습을 DevTools로 확인합니다
- 서버가 알아서 밀어 주는 **SSE**를 `SseEmitter`로 구현하고, 연결 한 줄로 여러 브라우저가 동시에 갱신되는 장면을 확인합니다
- 양방향 실시간이 필요한 **WebSocket(STOMP)**을 `@EnableWebSocketMessageBroker` + `/topic` · `/app`으로 얹어 pub/sub 흐름을 이해합니다
- 세 방식의 **통신 방향·서버 부하·지연·구현 난이도**를 비교표로 정리해 "어떤 상황에 어떤 걸 쓸지" 체크리스트를 얻습니다

이번 챕터는 "전화, 알림, 인터폰" 세 가지 도구를 같은 문제에 번갈아 써 보는 이야기입니다

챕터 5 마지막에서 동료가 남긴 한마디는 "사용자가 올린 다음에, 리사이즈 끝났는지 프런트는 어떻게 알아요?"였습니다. 3~5초 동안 화면이 비어 있을 수 없고, 그렇다고 사용자에게 수동 새로고침을 시킬 수도 없습니다. 팀장은 세 가지 방식을 이야기했습니다. 주기적으로 물어보거나, 서버가 밀어 주거나, 아예 양방향으로 열어 두거나. 이번 챕터는 그 세 가지를 한 챕터 안에서 다 만들어 보겠습니다. 같은 채팅 화면을 세 번 다시 구현하면서, 각 방식이 **어떤 장면에서 빛나고 어디서 무너지는지** 몸으로 체감합니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 준비

이번 챕터는 한 레포가 아니라 **세 개의 레포**를 차례로 다룹니다. 각 방식이 서로 다른 빌드 설정·컨트롤러 시그니처·프론트 코드를 쓰기 때문에, 한 프로젝트 안에 섞어 두면 비교 포인트가 흐려집니다. 세 개를 따로 열어 두고, 브라우저 탭도 셋으로 나눠 실행합니다.

레포	방식	주소
spring-polling	`setInterval`로 2초마다 GET	`github.com/metacoding-11-spring-reference/spring-polling`
spring-sse	`SseEmitter` + `text/event-stream`	`github.com/metacoding-11-spring-reference/spring-sse`
spring-websocket	STOMP + `/topic` · `/app`	`github.com/metacoding-11-spring-reference/spring-websoket`

터미널에서 셋을 한 상위 폴더 아래에 나란히 클론합니다.

터미널 실습 레포 클론

```
mkdir spring-realtime && cd spring-realtime
git clone https://github.com/metacoding-11-spring-reference/spring-polling.git
git clone https://github.com/metacoding-11-spring-reference/spring-sse.git
git clone https://github.com/metacoding-11-spring-reference/spring-websoket.git
```

각 레포의 파일 구조는 이렇습니다.

spring-polling 디렉토리

```
spring-polling/
├── build.gradle                                  # [참고] spring-boot-starter-web만
└── src/main/
    ├── java/com/metacding/polling/chat/
    │   ├── Chat.java                             # [참고] message 한 필드
    │   ├── ChatRequest.java                      # [참고] record(message)
    │   ├── ChatRepository.java                   # [참고] JpaRepository
    │   ├── ChatService.java                      # [실습] save / findAll(최신순)
    │   └── ChatController.java                   # [실습] GET /chats · POST /chats
```

```
    └── resources/templates/index.mustache       # [실습] setInterval(loadMessa
ges, 2000)
```

spring-sse 디렉토리

```
spring-sse/
├── build.gradle                                 # [참고] spring-boot-starter-w
eb만
└── src/main/
    ├── java/com/metacoding/sse/
    │   ├── config/SseEmitters.java              # [실습] Map<clientId, SseEmit
ter> 레지스트리
    │   └── chat/ChatController.java             # [실습] /chats/connect · POS
T /chats
    └── resources/templates/index.mustache       # [실습] new EventSource("/cha
ts/connect")
```

spring-websoket 디렉토리

```
spring-websoket/
├── build.gradle                                 # [실습] spring-boot-starter-w
ebsocket 추가
└── src/main/
    ├── java/com/metacoding/websocket/
    │   ├── config/WebSocketConfig.java          # [실습] @EnableWebSocketMessa
geBroker
    │   └── chat/ChatController.java             # [실습] @MessageMapping("/cha
ts")
    └── resources/templates/index.mustache       # [실습] Stomp.over(new WebSoc
ket(...))
```

> **이 챕터는 코드를 새로 작성하기보다 클론해서 따라 읽는 비중이 큽니다.** 세 방식의 핵심 차이를 **같은 채팅 기능**이라는 컨트롤 변수 위에 올려놓고 보는 게 이번 실습의 목적입니다. 각 레포의 TODO는 10줄 남짓입니다. 나머지 보일러플레이트(Entity·Repository·mustache HTML)는 미리 채워져 있습니다.

2. 실습 환경 확인

터미널 실습 환경 확인

```
java --version
./gradlew --version
```

세 레포 모두 Spring Boot 3.x · Java 17을 전제로 합니다. H2 인메모리 DB를 쓰기 때문에 DB 설치는 불필요합니다.

3. 사용할 구성 요소

이번 챕터에 등장하는 핵심 재료는 여섯 가지입니다.

재료	역할
`setInterval`	브라우저의 주기 타이머. 2초마다 `loadMessages()` 호출
`SseEmitter`	스프링이 제공하는 서버 → 클라이언트 단방향 스트림 객체
`EventSource`	브라우저 기본 API. `text/event-stream`을 받아 이벤트 디스패치
`@EnableWebSocketMessageBroker`	STOMP 메시지 브로커를 켜는 Spring 설정 어노테이션
`SimpMessagingTemplate`	서버에서 `/topic/**`으로 메시지를 발행하는 헬퍼
STOMP.js + SockJS	브라우저에서 WebSocket 위에 STOMP 프레임을 얹는 클라이언트 라이브러리

세 방식을 한 챕터에 몰아 두는 이유

실시간 통신은 공부할 땐 Polling·SSE·WebSocket을 각각 따로 배웁니다. 하지만 현업에서 실제로 쓰는 결정은 **"이 화면에서 무엇을 고를까"** 입니다. 이때 필요한 건 세 방식의 교과서적 정의가 아니라, 세 방식이 **같은 기능** 위에 올라갔을 때 어느 지점에서 어떻게 다른가를 직접 본 경험입니다. 연결이 유지되는지, 요청 수는 어떻게 찍히는지, 서버는 누가 어떻게 메시지를 밀어 주는지. 이 챕터는 그 세 가지 감각을 나란히 심는 걸 목표로 합니다.

4. 실습 순서

이번 챕터는 이 순서로 흘러갑니다.

2. `6.2`. 전화 걸기 방식 - Polling (`setInterval` + GET)
3. `6.3`. 전화기가 뜨거워지는 시간 - Polling의 비용
4. `6.4`. 서버가 알림을 밀어 준다 - SSE (`SseEmitter`)
5. `6.5`. 단방향의 한계 - 양방향이 필요한 순간
6. `6.6`. 인터폰 열기 - WebSocket (STOMP)
7. `6.7`. 세 방식 한 줄 비교와 선택 가이드

1에서 문제를 정리하고, 2~3에서 가장 단순한 Polling을 만들어 보겠습니다. 3에서 Polling의 비용이 눈에 보이면 4~5에서 SSE로 넘어갑니다. SSE가 단방향이라는 벽을 만나면 6에서 WebSocket을 엽니다. 7에서 세 방식을 비교하고 "어떤 때 어떤 걸 쓸지" 체크리스트를 뽑습니다.

6.1 업로드는 됐는데 끝난 걸 어떻게 알리나

챕터 5 끝의 동료 자리에서 시작합니다.

동료가 의자를 반 바퀴 돌렸습니다.

동료 : "올리는 건 이제 잘 돼요. 근데 이거 보세요. 업로드 끝나고 3초 동안 화면이 그냥 회색이에요."

오픈이는 동료의 모니터를 들여다봤습니다. 업로드 버튼을 누르면 스피너 하나가 돌다가, 어느 순간 멈추고, 그 다음에야 썸네일이 떴습니다. 그 사이의 **어느 순간**은 매번 달랐습니다. 빠르면 2초, 느리면 5초.

동료 : "사용자는 끝났는지 아닌지를 몰라요. 새로고침 한 번 누르면 뜨기는 하는데, 그걸 누가 해요."

"끝났다"를 서버가 말해 줘야 해.

오픈이는 수첩을 폈습니다. 지금 화면이 막힌 건 업로드 성능 문제가 아니었습니다. 문제는 **완료**라는 사건이 S3와 Lambda 쪽에서 벌어지는데, 브라우저는 그 사건을 모른다는 것이었습니다. 브라우저 입장에서는 PUT 업로드가 200을 받은 뒤로는 서버에서 아무 소식이 없었습니다.

오픈이 : "업로드가 끝난 걸 프런트가 어떻게 알게 할지, 그 채널을 하나 놔야 해요."

팀장이 뒤에서 커피를 들고 지나가다 멈췄습니다.

팀장 : "택배 조회를 생각해 봐요. 손님이 10초마다 택배사 홈페이지에 들어가서 F5를 누르면 그게 하나죠. 아니면 택배사 앱이 '지금 출발했습니다' 알림을 밀어 주는 것도 있고요. 아예 기사님이 인터폰으로 '지금 문 앞입니다' 이러는 것도 있잖아요."

오픈이 : "세 가지가 다 다르네요."

팀장 : "손님이 묻느냐, 택배사가 알려 주느냐, 아예 통화 중이냐. 셋 다 각자 쓰이는 자리가 달라요."

전화, 알림, 인터폰.

오픈이는 수첩에 세 단어를 나란히 적었습니다.

```
— 쪽지 —

1. 전화 (Polling)
   · 손님이 계속 묻기. 구현 쉬움. 빈 통화 많음

2. 알림 (SSE)
   · 가게가 알려 주기. 한 방향. 연결 하나 열어 둠

3. 인터폰 (WebSocket)
   · 양쪽 대화 가능. 연결 계속 유지. 세팅 번거로움
```

각각을 직접 만들어 봐야 감이 잡히겠다.

오픈이는 커피를 한 모금 마셨습니다. 세 방식을 **같은 기능** 위에 나란히 올려놓기로 했습니다. 업로드 완료 통지가 아니라, **채팅**으로요. 채팅은 "메시지가 생기면 상대 화면에 보여야 한다"는 같은 문제를 더 적나라하게 드러냅니다. 업로드 완료 통지도 본질은 같은 문제입니다. "어떤 사건이 서버 쪽에서 벌어지면, 그걸 브라우저가 알아야 한다."

키보드 소리만 또록또록 울리는 오후였습니다. 프로젝트 폴더를 세 개 나란히 만들고, 이름을 하나씩 붙였습니다. `spring-polling`, `spring-sse`, `spring-websoket`. 같은 채팅 기능을 세 번 다시 만드는, 조금 이상한 오후가 시작됐습니다.

6.2 전화 걸기 방식 - Polling

가장 먼저 열어 본 건 Polling입니다. 이유는 단순했습니다. 새로운 기술 한 줄을 배우지 않아도 구현이 끝나기 때문입니다. 기존 HTTP 위에서 `setInterval` 하나만 걸면 되는 구조입니다.

> **Polling.** 클라이언트가 일정 주기마다 서버에 "바뀐 거 있어요?"라고 반복 요청하는 방식. 서버가 상태 변화를 직접 알리는 수단이 없을 때 가장 단순하게 쓸 수 있다. 주기가 짧을수록 실시간성은 올라가지만 요청 수와 서버 부하가 비례해서 늘어난다.

6.2.1 Polling 전체 흐름

흐름을 먼저 머릿속에 넣고 코드를 봐야 합니다. 채팅 화면이 열리자마자 일어나는 일은 이렇습니다. 브라우저가 2초마다 서버에 "채팅 목록 주세요"라고 GET을 쏘고, 서버는 매번 DB에서 최신순으로 전체 목록을 뽑아 JSON으로 돌려줍니다. 다른 사람이 메시지를 보낸다 한들, **최대 2초 뒤**에야 내 화면에 뜹니다.

그림 6-1. 브라우저가 2초마다 서버에 "바뀐 거 있어요?"를 묻습니다

① 페이지 진입	`loadMessages()` 즉시 1회 호출 → 초기 목록 렌더
② 타이머 등록	`setInterval(loadMessages, 2000)` 2초 주기 반복
③ 매 주기 GET	`fetch("/chats")` → 서버가 DB에서 최신순 전체 목록 반환
④ 사용자 전송	`POST /chats` → DB 저장 → 즉시 `loadMessages()` 한 번 더
⑤ 최대 지연	상대 메시지는 최악의 경우 **2초 뒤**에 내 화면에 도착

서버는 상태 변화를 모릅니다. 오직 "물어볼 때만" 답합니다

이 구조를 코드로 구현해 보겠습니다.

6.2.2 Chat 저장과 전체 조회 서비스

Polling의 서버 쪽은 평범한 REST API입니다. 메시지 저장 한 건, 전체 목록 한 번. 서버가 "알림을 밀어줄" 책임이 없으므로 코드도 단순합니다.

`src/main/java/com/metacding/polling/chat/ChatService.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 1 src/main/java/com/metacding/polling/chat/ChatService.java. 저장·전체 조회

```
@RequiredArgsConstructor
@Service
public class ChatService {

    private final ChatRepository chatRepository;

    // TODO: 메시지 한 건을 저장한다
    @Transactional
    public Chat save(ChatRequest req) {
        Chat chat = Chat.builder()
                .message(req.message())
                .build();
        return chatRepository.save(chat);
    }

    // TODO: 전체 목록을 최신순으로 돌려준다
    public List<Chat> findAll() {
        Sort desc = Sort.by(Sort.Direction.DESC, "id");
        return chatRepository.findAll(desc);
    }
}
```

`save()`는 빌더로 엔티티를 만들어 JPA에 넘기는 평범한 저장 로직입니다. `findAll()`은 `Sort.Direction.DESC, "id"`로 **id 기준 내림차순**을 걸어, 가장 최근에 저장된 메시지가 배열의 앞쪽에 오도록 합니다. 프런트의 2초 타이머가 이 메서드를 매번 새로 호출합니다.

6.2.3 GET / POST 컨트롤러

`src/main/java/com/metacding/polling/chat/ChatController.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 2 src/main/java/com/metacding/polling/chat/ChatController.java. Polling 엔드포인트

```
@RequiredArgsConstructor
@Controller
public class ChatController {

    private final ChatService chatService;

    // TODO: 화면 진입용 페이지
```

```
    @GetMapping("/")
    public String index() {
        return "index";
    }

    // TODO: 메시지 저장
    @PostMapping("/chats")
    @ResponseBody
    public ResponseEntity<Chat> save(@RequestBody ChatRequest req) {
        Chat saved = chatService.save(req);
        return ResponseEntity.ok(saved);
    }

    // TODO: 전체 목록 (프런트 setInterval이 호출)
    @GetMapping("/chats")
    @ResponseBody
    public ResponseEntity<List<Chat>> list() {
        return ResponseEntity.ok(chatService.findAll());
    }
}
```

핵심은 `GET /chats` 입니다. 이 엔드포인트가 **2초마다 한 번씩** 브라우저에 호출됩니다. 서버는 이 요청이 "새 메시지가 있는지 확인하러 온" 호출인지, "그냥 한 번 보러 온" 호출인지 구분하지 않습니다. 매번 전체 목록을 다시 뽑아 돌려줍니다. 이 단순함이 Polling의 장점이자 약점입니다.

6.2.4 프런트 - setInterval로 주기 호출

`src/main/resources/templates/index.mustache` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 3 src/main/resources/templates/index.mustache. 2초 주기 폴링

```
// TODO: 서버에서 전체 목록을 가져와 화면을 갱신한다
async function loadMessages() {
  // 1. GET /chats 호출
  const res = await fetch("/chats");
  const data = await res.json();
  const box = document.getElementById("chat-box");

  // 2. 기존 리스트를 비우고 새 데이터로 다시 그린다
  box.innerHTML = "";
  data.forEach(chat => {
    const li = document.createElement("li");
    li.innerText = chat.message;
```

```
    box.appendChild(li);
  });
}

// TODO: 메시지 전송 후 즉시 한 번 갱신
async function sendMessage() {
  const messageInput = document.getElementById("message");
  const message = messageInput.value.trim();

  // 1. 서버에 저장 요청
  await fetch("/chats", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message })
  });

  // 2. 입력창 정리 후 바로 최신 목록 갱신
  messageInput.value = "";
  messageInput.focus();
  loadMessages();
}

// TODO: 초기 1회 + 2초마다 반복
loadMessages();
setInterval(loadMessages, 2000);
```

이 파일의 심장은 마지막 두 줄입니다. `loadMessages()`가 페이지 진입 직후 한 번, 그 이후에는 `setInterval(loadMessages, 2000)`이 **2초마다 한 번씩** 같은 함수를 다시 호출합니다. `sendMessage()` 안에서도 `loadMessages()`를 한 번 더 부르는 이유는, 내가 방금 보낸 메시지를 2초 기다리지 않고 곧바로 화면에 반영하기 위해서입니다.

Polling 구현에서 생각해 볼 것들

- **주기 선택**: 2초는 대화 화면에는 답답하고, 파일 업로드 완료 확인에는 과할 수 있습니다. 주기가 짧으면 실시간성이 올라가지만 서버 부하가 선형으로 증가합니다
- **전체 갱신의 비용**: 매번 `box.innerHTML = ""`로 리스트를 통째로 다시 그리고 있습니다. 메시지가 1000건이 되면 이 방식은 깨집니다. 실서비스에서는 "마지막으로 받은 id 이후만" 쿼리하는 방식으로 바뀝니다
- **서버 로그**: 이 코드가 돌아가는 동안 서버의 로그에는 `GET /chats 200`이 **2초마다 한 줄씩** 찍힙니다. 사용자 10명이 접속해 있으면 초당 5건. 사용자 100명이면 초당 50건

- **빈 응답 비율**: 실제로 메시지가 바뀌는 비율이 얼마나 될까요. 한 시간 동안 새 메시지가 10건 생긴다면, **2초 ×3600 = 1800번의 호출 중 10번만** 유효합니다

6.3 전화기가 뜨거워지는 시간

서버를 띄우고 브라우저를 엽니다.

터미널 실험 6-1 실행. Polling 서버 기동

```
cd spring-polling
./gradlew bootRun
```

`http://localhost:8080`에 접속하면 채팅 화면이 뜹니다. 메시지를 몇 개 입력해 보겠습니다.

[CAPTURE NEEDED: spring-polling 브라우저 화면. 상단에 입력창과 전송 버튼, 그 아래에 "안녕하세요", "반갑습니다", "2초 폴링 중" 같은 메시지가 최신순으로 쌓여 있는 상태]

얼핏 잘 돌아갑니다. 메시지를 보내면 바로 화면에 뜨고, 다른 브라우저 탭에서 메시지를 넣어도 내 쪽 화면에 최대 2초 안에 나타납니다.

그런데 F12로 DevTools를 열고 Network 탭을 보면 분위기가 달라집니다.

[CAPTURE NEEDED: Chrome DevTools의 Network 탭. "/chats" GET 요청이 2초 간격으로 끝없이 쌓이고 있는 화면. Status 200, Size 2B~100B 사이의 작은 응답이 타임라인에 줄줄이 찍혀 있는 모습]

이건 메시지 하나 없을 때도 계속 찍히잖아.

오픈이는 말 그대로 **내가 아무것도 안 하고 가만히 있는 동안에도** `GET /chats` 한 줄이 2초마다 빠짐없이 찍히는 걸 봤습니다. 한 시간이면 1800건. 10명이 접속 중이면 18000건. 그중 대부분은 **직전 요청과 완전히 같은 데이터를 받아 오는** 빈 호출입니다.

서버 로그도 같은 모양이었습니다.

terminal - spring-polling 서버 로그

```
2026-04-24 14:02:10.112 INFO GET /chats 200 (2ms)
2026-04-24 14:02:12.115 INFO GET /chats 200 (2ms)
2026-04-24 14:02:14.118 INFO GET /chats 200 (2ms)
2026-04-24 14:02:16.121 INFO GET /chats 200 (2ms)
2026-04-24 14:02:18.124 INFO GET /chats 200 (2ms)
... (한 시간에 1800건) ...
2026-04-24 15:02:04.811 INFO GET /chats 200 (2ms)
```

같은 응답이 2초마다. 유저가 아무것도 안 해도 서버는 계속 일합니다

동료가 모니터 너머로 화면을 들여다봤습니다.

동료 : "저 2초짜리 반복, 사용자 1000명 붙으면 초당 500건이에요."

오픈이 : "그중 거의 전부가 '변한 거 없음'이고요."

팀장 : "서버가 물어 봐서 답하는 구조면 그래요. 묻는 사람이 많아지면 답하는 사람이 바빠지죠."

서버가 먼저 말하게 해야 해.

업로드 완료 통지에 Polling을 붙이면 더 이상합니다. 업로드는 한 번 일어나고 끝입니다. 그런데 서버는 그 한 번을 위해 **모든 업로드 요청 뒤로 3~5초 동안 계속** `/status/{id}` **를 받아 내야** 합니다. 대부분이 "아직 안 끝났어요"라는 빈 응답으로 버려집니다.

필요한 건 반대 방향의 채널이었습니다. 상태가 **바뀔 때** 서버가 먼저 밀어 주는 연결.

6.4 서버가 알림을 밀어 준다 - SSE

오픈이는 두 번째 폴더 `spring-sse`를 열었습니다.

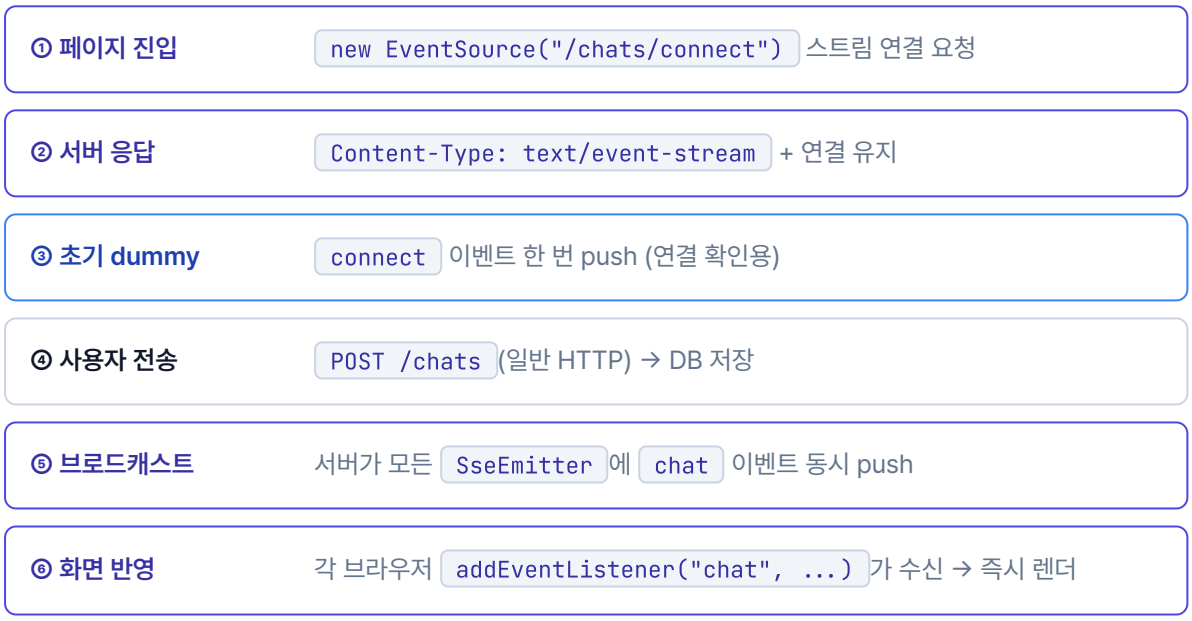
> **SSE (Server-Sent Events)**. 클라이언트가 한 번 연결을 열어 두면, 서버가 그 연결을 통해 이벤트를 **계속 밀어 주는** 단방향 스트리밍 프로토콜. HTTP 위에서 동작하며 `Content-Type: text/event-stream`을 사용한다. 브라우저는 `EventSource` API로 받는다. 서버 → 클라이언트 한 방향만 가능하고, 클라이언트가 서버로 보내려면 별도의 HTTP 요청을 쓴다.

팀장 : "택배가 이제 출발했어요. 이런 걸 앱이 알림으로 띄워 주잖아요. 그때 손님은 앱한테 '지금 출발했어?'를 계속 묻지 않아요."

SSE의 모양이 정확히 그렇습니다. 브라우저는 `new EventSource("/chats/connect")` 한 줄로 서버와 스트림 연결을 엽니다. 그 뒤로 서버가 `emitter.send(...)`를 호출할 때마다, 브라우저에 이벤트가 **바로** 도착합니다. 브라우저는 아무 요청도 보내지 않고 듣기만 합니다.

6.4.1 SSE 전체 흐름

그림 6-2. 한 번 연결을 열어 두면 서버가 알아서 밀어 줍니다



6.4.2 SseEmitters - 연결 레지스트리

SSE 서버의 핵심은 **지금 누가 연결되어 있는가를 보관하는 레지스트리**입니다. 연결이 열릴 때 `SseEmitter`를 Map에 저장하고, 메시지가 저장될 때 그 Map을 순회하면서 모두에게 `send()`를 호출합니다.

`src/main/java/com/metacoding/sse/config/SseEmitters.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 4 src/main/java/com/metacoding/sse/config/SseEmitters.java. 연결 레지스트리

```java
@Slf4j
@Component
public class SseEmitters {

    // TODO: 접속 중인 클라이언트 보관소
    private final Map<String, SseEmitter> emitters = new ConcurrentHashMap<>();

    // TODO: 새 연결을 등록한다 (중복 연결은 이전 것을 닫는다)
    public SseEmitter add(String clientId, SseEmitter emitter) {
        // 1. 같은 clientId가 이미 있으면 이전 연결을 정상 종료
        if (emitters.containsKey(clientId)) {
            SseEmitter prevEmitter = emitters.get(clientId);
            prevEmitter.complete();
        }

        // 2. 새 emitter를 Map에 보관
        emitters.put(clientId, emitter);

        // 3. 연결이 끝나면 Map에서 제거되도록 콜백 등록
        final SseEmitter currentEmitter = emitter;
        emitter.onCompletion(() -> emitters.remove(clientId, currentEmitter));
        emitter.onTimeout(() -> emitter.complete());
        emitter.onError((e) -> emitter.complete());

        return emitter;
    }

    // TODO: 현재 연결된 모든 클라이언트에게 이벤트를 밀어 보낸다
    public void sendAll(Chat chat) {
        emitters.entrySet().forEach(entry -> {
            String clientId = entry.getKey();
            SseEmitter emitter = entry.getValue();

            try {
                // 4. "chat" 이벤트 이름 + Chat 객체를 data로 push
                emitter.send(SseEmitter.event()
                        .name("chat")
                        .data(chat));
            } catch (Exception e) {
                // 5. 전송 실패 시 해당 연결 정리
                emitter.complete();
                emitters.remove(clientId);
            }
        });
    }
}
```

이 클래스에는 두 가지 중요한 결정이 있습니다. 첫째, **저장소로 `ConcurrentHashMap`을 씁니다.** SSE는 동시에 여러 스레드가 `add`로 등록하고 `sendAll`로 순회합니다. 일반 `HashMap`은 이 상황에서 내부 구조가 꼬일 수 있습니다. `ConcurrentHashMap`은 동시 접근에서도 스레드 안전합니다.

둘째, `onCompletion` · `onTimeout` · `onError` **세 콜백으로 청소까지 맡깁니다.** 한 번 등록한 emitter를 계속 Map에 들고 있으면, 연결이 끊긴 뒤에도 `sendAll`이 죽은 emitter에 계속 쏘려다가 예외를 던집니다. 끝난 연결은 Map에서 자동으로 빠져야 합니다.

```
— 실험 메모 —

1. 왜 ConcurrentHashMap?
   · add/remove/iterate가 서로 다른 스레드에서 동시에 일어남
   · HashMap은 내부 bucket 리사이즈 중 다른 스레드가 접근하면 무한루프·예외 발생 가능

2. 왜 onCompletion에서 remove?
   · 연결이 끊긴 뒤에도 Map에 남아 있으면 다음 sendAll에서 죽은 연결에 쏨
   · 죽은 연결 누적 → 메모리 누수 + 매 전송마다 예외 비용
```

6.4.3 SSE 연결 엔드포인트

연결을 열고, 레지스트리에 등록하고, 초기 dummy 이벤트 한 번 쏘는 흐름입니다.

`src/main/java/com/metacoding/sse/chat/ChatController.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 5 src/main/java/com/metacoding/sse/chat/ChatController.java. SSE 연결 + 브로드캐스트

```
@RequiredArgsConstructor
@Controller
public class ChatController {

    private final ChatService chatService;
    private final SseEmitters sseEmitters;
    private final HttpSession session;

    // TODO: 브라우저가 EventSource로 여는 스트림 연결 엔드포인트
    @GetMapping(value = "/chats/connect", produces = MediaType.TEXT_EVENT_STREAM_V
ALUE)
    public ResponseEntity<SseEmitter> connect() {
```

```
        // 1. 세션 id를 clientId로 사용
        String clientId = session.getId();

        // 2. 60초 타임아웃의 emitter 생성
        SseEmitter emitter = new SseEmitter(60 * 1000L);

        // 3. 레지스트리에 등록
        sseEmitters.add(clientId, emitter);

        // 4. 연결 확인용 dummy 이벤트 1회 (연결 유지 + 재연결 시 403 방지)
        try {
            emitter.send(SseEmitter.event().name("connect").data("connected"));
        } catch (Exception e) {
            emitter.complete();
        }

        return ResponseEntity.ok(emitter);
    }

    // TODO: 메시지 저장 + 모든 연결에 브로드캐스트
    @PostMapping("/chats")
    @ResponseBody
    public Chat save(@RequestBody ChatRequest req) {
        // 5. DB 저장
        Chat saved = chatService.save(req);
        // 6. 현재 연결된 모든 브라우저에 즉시 push
        sseEmitters.sendAll(saved);
        return saved;
    }

    // TODO: 초기 진입용 전체 목록
    @GetMapping("/chats")
    @ResponseBody
    public List<Chat> list() {
        return chatService.findAll();
    }
}
```

핵심은 두 지점입니다. `@GetMapping`의 `produces = MediaType.TEXT_EVENT_STREAM_VALUE`가 응답 헤더를 `text/event-stream`으로 설정해 스프링이 연결을 **끊지 않고** 유지하게 만듭니다. 그리고 `save()` 안의 `sseEmitters.sendAll(saved)` 한 줄이 이 챕터의 심장입니다. 저장 직후, 현재 연결된 모든 브라우저에 같은 Chat 객체를 즉시 밀어 줍니다.

`connect()` 메서드 안에서 dummy 이벤트를 한 번 보내는 이유는 별도 설명이 필요합니다. 브라우저 `EventSource`는 일정 시간 동안 서버로부터 **아무 데이터도 받지 않으면 재연결을 시도**합니다. 재연결 시 세션/CSRF 검증을 다시 받게 되고, 이 과정에서 403이 떨어지는 경우가 생깁니다. 연결 직후 dummy 이벤트를 한 번 쏴 두면, "방금 뭔가 받았으니 재연결 안 해도 된다"는 신호가 되어 안정적으로 연결이 유지됩니다.

6.4.4 프런트 - EventSource로 듣기

`src/main/resources/templates/index.mustache`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 6 src/main/resources/templates/index.mustache. EventSource로 이벤트 수신

```
// TODO: SSE 스트림 연결을 연다
const sse = new EventSource("/chats/connect");

// TODO: 연결 확인 이벤트 수신
sse.addEventListener("connect", (e) => {
  console.log("connected:", e.data);
});

// TODO: 서버가 sendAll()로 밀어 주는 chat 이벤트 수신
sse.addEventListener("chat", (e) => {
  // 1. e.data는 문자열. Chat 객체로 파싱
  const chat = JSON.parse(e.data);
  // 2. 화면 상단에 prepend (최신이 위로)
  appendChat(chat.message);
});

// TODO: 메시지 전송은 일반 HTTP POST (SSE는 서버 → 클라 단방향이므로)
async function sendMessage() {
  const messageInput = document.getElementById("message");
  const message = messageInput.value.trim();

  await fetch("/chats", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message })
  });

  messageInput.value = "";
  messageInput.focus();
}
```

```
// TODO: 수신한 메시지를 화면에 추가
function appendChat(message) {
  const box = document.getElementById("chat-box");
  const li = document.createElement("li");
  li.innerText = message;
  box.prepend(li);
}
```

이 파일에는 `setInterval`이 없습니다. **한 줄도 없습니다.** `new EventSource("/chats/connect")` 한 줄이 연결을 열면, 그 뒤로 브라우저는 그저 듣기만 합니다. `sse.addEventListener("chat", ...)`가 서버가 `emitter.send(...name("chat")...)`로 보낸 이벤트를 받아 `appendChat`에 넘깁니다.

한 가지 짚어 둘 점은 **전송은 여전히 HTTP POST**라는 것입니다. SSE는 서버 → 클라이언트 한 방향만 열어 두는 프로토콜입니다. 클라이언트가 서버로 뭔가 보내야 하면 기존 HTTP 요청을 그대로 씁니다. 이 구조가 SSE의 깔끔함이자 제약입니다. "알림"만 필요한 곳에는 완벽하게 맞지만, "대화"가 필요한 곳에는 절반만 맞습니다.

6.4.5 실행과 확인

터미널 실험 6-2 실행. SSE 서버 기동

```
cd spring-sse
./gradlew bootRun
```

`http://localhost:8080`을 **두 개 탭**으로 엽니다. 한 쪽에서 메시지를 보내면 다른 쪽에도 즉시 (폴링 지연 없이) 뜨는지 확인합니다.

[CAPTURE NEEDED: spring-sse 브라우저 두 탭을 나란히 열어, 왼쪽 탭에서 "안녕"을 입력하자 오른쪽 탭의 chat-box 상단에도 "안녕"이 즉시 나타난 화면]

DevTools의 Network 탭에서 `chats/connect` 요청을 선택하면, EventStream이라는 특별한 탭이 뜹니다. 거기에 서버가 보낸 이벤트가 시간순으로 쌓입니다.

[CAPTURE NEEDED: Chrome DevTools에서 /chats/connect 요청을 선택 → EventStream 탭. Type=connect(data=connected), Type=chat(data={"id":1,"message":"안녕"}), Type=chat(data={"id":2,"message":"반갑습니다"})가 시간순으로 나열된 화면]

Polling과 가장 큰 차이가 여기서 보입니다. `/chats` 같은 GET 요청이 **2초마다** 찍히지 않습니다. `/chats/connect` 단 한 건만이 "Pending" 상태로 연결을 유지하고, 그 연결 안에서 이벤트만 오갑니다.

그림 6-3. 같은 화면, Polling vs SSE의 네트워크 풍경



SSE는 "알림"에 최적. 양방향 대화가 필요하면 이야기가 달라집니다

SSE 구현에서 생각해 볼 것들

- **타임아웃 60초**: `new SseEmitter(60 * 1000L)`은 60초 동안 아무 이벤트가 없으면 서버가 연결을 정리한다는 뜻입니다. 실서비스에서는 하트비트(주기적 ping)를 같이 쏴 두거나 타임아웃을 길게 잡습니다
- **clientId 기준 중복 연결 처리**: 같은 세션으로 두 번 접속하면 이전 연결을 `complete()`로 닫고 새 걸로 교체합니다. 중복 연결을 방치하면 한 사용자에게 같은 이벤트가 두 번 갑니다
- **프록시·로드밸런서**: nginx·ALB 같은 중간 장비가 idle 타임아웃으로 연결을 끊을 수 있습니다. SSE를 쓸 때는 해당 장비의 `proxy_read_timeout`·`Idle timeout`을 충분히 크게 설정해야 합니다
- **scale-out 문제**: 서버를 두 대로 늘리면, A 서버에 연결된 브라우저에 B 서버에서 발행한 이벤트를 직접 보낼 수 없습니다. Redis Pub/Sub 같은 공용 브로커로 서버 간 이벤트를 전파하는 구조가 필요합니다

6.5 단방향의 한계

SSE를 붙이고 나서 동료가 모니터 너머로 다시 왔습니다.

동료 : "알림은 이제 잘 와요. 근데 이거 질문 하나요. 만약 채팅 앱이면, 상대가 **타이핑 중**이라는 표시도 띄워야 하잖아요. 그거는 서버로 보내야 하는 거 아니에요?"

오픈이 : "맞아요. '타이핑 중'은 클라 → 서버로 한 번 보내고, 서버가 다른 사람한테 밀어 주는 흐름이에요."

동료 : "그럼 보낼 때는 POST로 쏘고, 받을 때는 SSE로 듣고요?"

오픈이가 고개를 끄덕였습니다. 그런데 말을 하면서 조금 이상해졌습니다. '타이핑 중'은 초당 수십 번 찍힙니다. 그걸 매번 HTTP POST로 쏘면, POST 요청 자체가 업로드 폴링만큼이나 자주 발생합니다. 연결 핸드셰이크 비용이 매번 붙고, 쿠키·헤더가 매번 같이 갑니다. 단방향 채널 하나만으로는 부족했습니다.

팀장 : "방문자 센터랑 택배 기사님이 인터폰으로 대화하는 거랑, 방문자 센터에서 문자를 일방적으로 보내 주는 거랑, 다르잖아요. 양쪽이 실시간으로 말을 주고받아야 하는 자리가 있어요."

알림은 SSE, 대화는 WebSocket.

채팅처럼 **양쪽이 자주 주고받는** 상황, 협업 편집처럼 **한 쪽의 작은 변경이 즉시 상대에게 반영돼야 하는** 상황, 게임처럼 **지연이 밀리초 단위로 문제가 되는** 상황. 이런 자리에는 SSE가 아니라 WebSocket이 맞습니다.

6.6 인터폰 열기 - WebSocket

세 번째 폴더 `spring-websoket`을 엽니다.

> **WebSocket**. HTTP로 최초 핸드셰이크를 한 뒤, 같은 TCP 연결을 **양방향 프레임 교환 채널**로 업그레이드하는 프로토콜. `ws://` 또는 `wss://` 스킴을 쓴다. 한 번 연결되면 끊기 전까지 양쪽이 자유롭게 메시지를 보낼 수 있다.
>
> **STOMP (Simple Text Oriented Messaging Protocol)**. WebSocket 위에 **목적지(destination) 기반 pub/sub** 의미를 얹는 메시징 규약. `SUBSCRIBE /topic/chats`로 구독하고 `SEND /app/chats`로 발행한다. 스프링은 `@EnableWebSocketMessageBroker`와 `@MessageMapping`으로 이 규약을 편하게 구현하도록 지원한다.

6.6.1 두 가지 흐름 - 브로커 직행 vs 컨트롤러 경유

WebSocket+STOMP에는 메시지가 서버를 거치는 방식이 두 가지 있습니다. 먼저 두 가지를 머릿속에 넣고 코드를 봐야 실습 구조가 읽힙니다.

그림 6-4. 흐름 A) 브로커 직행과 흐름 B) 컨트롤러 경유

흐름 A · 브로커 직행

/topic/** 클라이언트 → `/topic/chats`로 직접 SEND → 브로커가 구독자 전원에게 즉시 브로드캐스트. 서버 컨트롤러 경유 없음. 저장·검증이 필요 없는 단순 신호(타이핑 표시, 커서 위치)에 적합.

흐름 B · 컨트롤러 경유

/app/** → **/topic/**** 클라이언트 → `/app/chats`로 SEND → `@MessageMapping` 메서드가 받아 DB 저장·검증·가공 → `SimpMessagingTemplate.convertAndSend("/topic/chats", ...)`로 발행 → 브로커가 구독자 전원에게 브로드캐스트. 저장·검증이 필요한 일반 채팅에 적합.

이번 실습은 B(컨트롤러 경유)를 사용합니다. 저장이 필요하기 때문입니다

6.6.2 의존성과 WebSocket 설정

`build.gradle`을 엽니다.

실습 7 build.gradle. WebSocket 스타터 추가

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    // TODO: WebSocket(STOMP 포함) 스타터 추가
    implementation 'org.springframework.boot:spring-boot-starter-websocket'
    // ... H2, JPA 등은 미리 들어 있음
}
```

스타터 하나로 Spring WebSocket·STOMP·Simple Broker가 모두 들어옵니다. 그 위에 설정 클래스가 필요합니다.

`src/main/java/com/metacoding/websocket/config/WebSocketConfig.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 8 src/main/java/com/metacoding/websocket/config/WebSocketConfig.java. STOMP 엔드포인트·브로커 설정

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    // TODO: 브라우저가 WebSocket 연결을 맺는 엔드포인트 등록
    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        // 1. /ws 경로로 연결 (브라우저는 ws://localhost:8080/ws 로 접속)
        registry.addEndpoint("/ws")
                .setAllowedOriginPatterns("*");
    }

    // TODO: 메시지 라우팅 규칙 설정
    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        // 2. /app 프리픽스는 서버 @MessageMapping으로 라우팅 (흐름 B)
        registry.setApplicationDestinationPrefixes("/app");

        // 3. /topic 프리픽스는 내장 Simple Broker가 처리 (구독 채널)
        registry.enableSimpleBroker("/topic");
    }
}
```

세 줄의 설정이 WebSocket의 라우팅 지도 전체입니다. `/ws`는 **연결을 맺는 문**이고, `/app`은 **서버 컨트롤러로 들어가는 길**이고, `/topic`은 **브로커가 중계하는 구독 채널**입니다. 클라이언트가 `SEND /app/chats`로 보내면 서버의 `@MessageMapping("/chats")` 메서드가 받고, 처리 결과를 `/topic/chats`로 밀어내면 구독자 전원에게 브로드캐스트됩니다.

6.6.3 메시지 핸들러

`src/main/java/com/metacoding/websocket/chat/ChatController.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 9 src/main/java/com/metacoding/websocket/chat/ChatController.java. @MessageMapping으로 수신 + /topic으로 발행

```
@RequiredArgsConstructor
@Controller
```

```
public class ChatController {

    private final ChatService chatService;
    private final SimpMessagingTemplate messagingTemplate;

    // TODO: 클라이언트가 /app/chats로 보낸 메시지를 받아 처리한다
    @MessageMapping("/chats")
    public void handle(ChatRequest payload) {
        // 1. DB 저장
        Chat saved = chatService.save(payload);

        // 2. /topic/chats 구독자 전원에게 브로드캐스트
        messagingTemplate.convertAndSend("/topic/chats", saved);
    }
}
```

여기서 한 가지 혼동 포인트가 있습니다. `@MessageMapping("/chats")`인데 클라이언트는 `/app/chats`로 보낸다는 점입니다. 이유는 `WebSocketConfig`의 `setApplicationDestinationPrefixes("/app")` 설정 때문입니다. `/app`은 **설정에서 자동으로 붙는 프리픽스**이므로, 컨트롤러 코드에서는 그 뒤의 `/chats`만 적습니다. 클라이언트가 보낼 때는 프리픽스를 포함해서 `/app/chats`로 보냅니다.

`SimpMessagingTemplate.convertAndSend(...)`는 서버에서 임의의 목적지로 메시지를 발행할 때 쓰는 헬퍼입니다. 이 한 줄이 브로커에게 "이 메시지를 `/topic/chats` 구독자 전원에게 돌려라"라고 명령합니다. 브로커는 내부적으로 해당 토픽의 구독자 리스트를 들고 있다가 순회하면서 각자의 WebSocket 연결로 프레임을 내려보냅니다.

6.6.4 프런트 - STOMP.js로 연결·구독·전송

`src/main/resources/templates/index.mustache`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 10 src/main/resources/templates/index.mustache. STOMP 클라이언트

```
// TODO: WebSocket + STOMP 연결 객체 준비
const socketUrl = (location.protocol === "https:" ? "wss://" : "ws://") + locatio
n.host + "/ws";
const stompClient = Stomp.over(new WebSocket(socketUrl));

// TODO: 연결 완료 후 /topic/chats 구독
stompClient.connect({}, () => {
```

```
  stompClient.subscribe("/topic/chats", (msg) => {
    // 1. 서버가 /topic/chats로 발행한 메시지를 수신
    const data = JSON.parse(msg.body);
    appendChat(data.message);
  });
});

// TODO: 전송은 /app/chats로 SEND
function sendMessage() {
  const messageInput = document.getElementById("message");
  const message = messageInput.value.trim();

  // 2. 연결 상태 확인 (아직 핸드셰이크 중일 수 있음)
  if (!stompClient.connected) {
    alert("연결 중입니다. 잠시 후 다시 시도하세요.");
    return;
  }

  // 3. SEND /app/chats (서버 @MessageMapping으로 라우팅됨)
  stompClient.send("/app/chats", {}, JSON.stringify({ message }));
  messageInput.value = "";
  messageInput.focus();
}

// TODO: 수신 메시지를 화면 상단에 prepend
function appendChat(message) {
  const box = document.getElementById("chat-box");
  const li = document.createElement("li");
  li.innerText = message;
  box.prepend(li);
}
```

코드 구조는 SSE와 비슷하지만 한 가지가 근본적으로 다릅니다. `fetch` **같은 HTTP 요청이 한 줄도 없습니다.** 전송도 `stompClient.send`, 수신도 `stompClient.subscribe`입니다. 같은 `/ws` 연결 하나가 양방향으로 다 처리합니다.

`stompClient.connected` 체크는 실무에서 꼭 필요한 방어선입니다. 페이지 진입 직후에는 아직 핸드셰이크가 끝나지 않아 `connected`가 false일 수 있습니다. 사용자가 그 순간에 전송 버튼을 누르면 메시지가 유실됩니다.

6.6.5 실행과 확인

터미널 실험 6-3 실행. WebSocket 서버 기동

```
cd spring-websoket
./gradlew bootRun
```

`http://localhost:8080`을 두 탭으로 엽니다. 어느 쪽에서 메시지를 보내든, 양쪽 화면에 거의 동시에 뜹니다.

[CAPTURE NEEDED: spring-websoket 브라우저 두 탭. 왼쪽에서 "안녕"을 치자마자 오른쪽 chat-box 상단에 "안녕"이 뜨고, 오른쪽에서 "반갑습니다"를 치자 왼쪽에도 즉시 반영된 화면]

DevTools의 Network 탭에서 WS 필터를 걸면 `ws` 요청 한 줄이 보입니다. 그걸 선택하면 Frames 탭이 뜹니다. 거기에 STOMP 프레임이 방향(보냄/받음)별로 시간순 나열됩니다.

[CAPTURE NEEDED: Chrome DevTools에서 /ws WS 연결의 Messages(또는 Frames) 탭. CONNECT · CONNECTED · SUBSCRIBE /topic/chats · SEND /app/chats · MESSAGE /topic/chats 프레임이 시간순으로 나열된 화면]

프레임을 하나씩 읽어 보면 STOMP의 의미가 드러납니다. 연결이 맺어지면 `CONNECT` → `CONNECTED`가 오가고, 브라우저가 `SUBSCRIBE /topic/chats`로 구독을 선언합니다. 사용자가 메시지를 보내면 `SEND /app/chats` 프레임이 서버로 올라가고, 서버가 처리한 뒤 `MESSAGE /topic/chats` 프레임이 모든 구독자에게 내려옵니다.

WebSocket 구현에서 생각해 볼 것들

- **연결 상태 관리**: 연결이 끊겼을 때의 재연결 로직, 재연결 이후 구독 복구, 메시지 유실 처리가 운영의 절반입니다. `stompClient.connected`만 믿으면 안 됩니다
- **인증·권한**: WebSocket은 한 번 연결되면 그 연결 안에서 인증 정보가 유지됩니다. 토큰 만료·권한 변경을 중간에 반영하려면 별도 이벤트로 재인증을 강제하거나 연결을 끊어야 합니다
- **scale-out (스티키 세션)**: 서버를 여러 대로 늘리면 한 사용자의 WebSocket 연결이 A 서버에 묶입니다. 로드밸런서의 스티키 세션 설정이 필요하고, 서버 간 메시지 전파에는 외부 브로커(Redis·RabbitMQ·Kafka) 연동이 뒤따릅니다
- **브로커 선택**: 이번 실습의 `enableSimpleBroker`는 Spring 내장 인메모리 브로커입니다. 서버 한 대에서만 의미가 있습니다. 여러 서버로 확장하거나 메시지 영속성이 필요하면 `enableStompBrokerRelay`로 RabbitMQ/ActiveMQ 같은 외부 브로커를 붙입니다

- **흐름 A vs B 선택**: 저장·검증·가공이 필요하면 컨트롤러 경유(B)가 맞습니다. "타이핑 중" 같은 저장 없는 신호는 브로커 직행(A)이 낫습니다

6.7 세 방식 한 줄 비교와 선택 가이드

세 개의 터미널과 세 개의 브라우저 탭을 앞에 두고, 오픈이는 수첩을 다시 폈습니다.

같은 채팅을 세 번 다시 만들고 나니, 각각이 어디에 맞는지 감이 잡힌다.

그림 6-5. Polling · SSE · WebSocket 한눈 비교

방식	Polling · SSE · WebSocket
통신 방향	Polling: 요청/응답 반복 · SSE: 서버 → 클라(단방향) · WebSocket: 양방향
연결	Polling: 매 요청마다 · SSE: 1회 연결 유지 · WebSocket: 1회 연결 유지
지연	Polling: 폴링 주기 영향 · SSE: 거의 실시간 · WebSocket: 실시간
구현 난이도	Polling: 낮음 · SSE: 중간 · WebSocket: 높음
서버 부하	Polling: 사용자 수 × 주기에 비례 · SSE: 이벤트 수에 비례 · WebSocket: 이벤트 수 + 연결 유지 비용
브라우저 지원	Polling: 전부 · SSE: 모던 브라우저 · WebSocket: 모던 브라우저(IE 제외)
재연결	Polling: 필요 없음 · SSE: EventSource가 자동 재시도 · WebSocket: 직접 구현 필요
적합한 자리	Polling: 완료 확인·단발성 상태 · SSE: 알림·진행률·스트림 이벤트 · WebSocket: 채팅·협업·양방향 제어

같은 채팅을 세 번 구현하고 나서야 보이는 차이입니다

이 표를 수첩에 옮기고 나서, 오픈이는 챕터 5에서 남아 있던 문제로 돌아왔습니다. **업로드 완료 통지**에는 세 가지 중 어느 게 맞을까요.

업로드는 한 번 일어나고 끝나는 사건입니다. 사용자는 "아직이에요?"를 묻고 싶은 게 아니라, "끝났다"는 알림을 듣고 싶은 것입니다. 클라이언트가 서버로 보낼 것도 없습니다. S3가 객체를 받고, Lambda가 썸네일을 만들고, 서버가 DB에 한 행을 적었다는 사실만 전달되면 됩니다. **단방향**, **가끔 발생하는 사건**, **대화 아님**.

SSE가 맞다.

업로드 완료뿐 아니라, **진행률 표시**(`10%` → `50%` → `90%` → 완료), **배포 로그 스트림**, **주식 시세 알림** 같은 자리도 같은 성격을 갖습니다. 서버에서만 이벤트가 발생하고, 클라는 듣기만 하면 됩니다. SSE 한 방향으로 충분합니다.

반면 **채팅**처럼 사용자가 말을 보내고 상대가 답하는 자리, **협업 편집**처럼 한 사람이 셀을 편집하면 모두가 실시간으로 그 변화를 봐야 하는 자리, **실시간 게임**처럼 지연이 밀리초 단위로 문제가 되는 자리는 WebSocket 아니면 답이 없습니다. 양방향 채널을 한 번 열어 두고, 그 위에서 프레임을 주고받습니다.

Polling은 이번 챕터에서는 "문제 제기"의 역할이었지만, 실무에서 여전히 현역입니다. **단순한 상태 조회**(주문 상태, 결제 완료 여부), **10분에 한 번 정도면 충분한 새로 고침**(관리자 대시보드), **브라우저 호환성이 중요한 구식 환경** 같은 자리에는 Polling이 가장 간결한 선택입니다. 서버에 별도 인프라가 필요 없고, 코드가 `setInterval` 한 줄이면 됩니다.

이것만은 기억하자

- → **Polling은 "묻는 쪽이 주도"**. 구현이 가장 단순하지만 변화가 없어도 계속 요청이 발생한다. 폴링 주기 = 최대 지연 = 서버 부하 계수
- → **SSE는 "서버가 밀어 주는 단방향"**. `SseEmitter` + `text/event-stream`으로 한 연결에서 이벤트만 스트리밍. 알림·진행률·배포 로그처럼 **서버 → 클라 방향만** 필요한 자리에 맞다
- → **WebSocket(STOMP)은 "열어 두고 양쪽이 대화"**. `@EnableWebSocketMessageBroker` + `/app` · `/topic` 프리픽스로 pub/sub 구조. 채팅·협업·실시간 제어처럼 **양방향**이 필수인 자리에만 쓴다

→ **선택의 축은 두 개**. 방향(단방향/양방향)과 빈도(가끔/지속). 가끔·단방향이면 Polling, 자주·단방향이면 SSE, 자주·양방향이면 WebSocket

→ **업로드 완료 통지의 답**. SSE. 사건이 가끔 발생하고, 서버가 알리기만 하면 되는 자리

동료가 뒤에서 모니터를 들여다봤습니다.

동료 : "그럼 업로드 완료는 SSE로 가는 걸로요. 근데요, 진짜 사용자가 수천 명이 동시에 접속하는 서비스는요? 저 SseEmitter Map이 수천 개가 되면 괜찮을까요."

오픈이 : "그건 서버 한 대일 때 이야기예요. 여러 대면 Map이 나뉘어 있잖아요. A 서버에 붙은 사용자한테 B 서버에서 일어난 이벤트는 어떻게 가요."

팀장 : "그래서 라이브 스트리밍 서비스들은 브로커를 중간에 두죠. Redis Pub/Sub이든, RabbitMQ든, Kafka든."

연결 하나 여는 건 쉬웠다. 그 연결을 **수천 개** 유지하는 건 다른 문제다.

오픈이는 수첩을 덮었습니다. 한 서버 위에서는 세 방식 다 배웠고 선택 기준도 정리됐습니다. 남은 질문은 **스케일**입니다. 사용자 수가 열 명에서 천 명으로, 만 명으로 늘어나면 같은 코드가 그대로 굴러갈까요. 그 질문의 답을 찾는 것이 다음 챕터의 시작입니다.

용어 정리

이야기 속 표현	진짜 용어	정식 정의
전화로 반복해서 묻기	Polling	클라이언트가 일정 주기로 서버에 상태를 재요청하는 통신 패턴. HTTP 기반으로 어디서든 동작하지만, 변화가 없어도 요청이 계속 발생해 서버 부하와 지연(최대 = 폴링 주기)이 발생
2초 타이머	`setInterval(fn, ms)`	브라우저 내장 함수. 지정한 밀리초 간격으로 콜백을 반복 호출. 클리어에는 `clearInterval` 사용

알림 앱의 푸시	SSE (Server-Sent Events)	HTTP 기반 단방향 스트리밍. `Content-Type: text/event-stream`으로 서버가 이벤트를 연속 전송. 브라우저는 `EventSource` API로 수신하며 자동 재연결 지원
스트림 연결 객체	`SseEmitter`	스프링 제공 객체. 서버 측 SSE 연결을 나타내며 `send()`·`complete()`·`onCompletion()` 등으로 수명 주기 제어
브라우저의 수신 API	`EventSource`	브라우저 표준 API. `new EventSource(url)`로 SSE 연결을 열고 `addEventListener(name, handler)`로 이벤트별 핸들러 등록
연결 목록을 안전하게 들고 있는 자료구조	`ConcurrentHashMap`	자바의 스레드 안전 해시맵. 여러 스레드가 동시에 put·get·iterate를 호출해도 내부 무결성이 유지됨
인터폰 (양방향 실시간 채널)	WebSocket	HTTP로 최초 핸드셰이크 후 같은 TCP 연결을 양방향 프레임 교환 채널로 업그레이드하는 프로토콜. `ws://` / `wss://` 스킴 사용
목적지 기반 대화 규약	STOMP (Simple Text Oriented Messaging Protocol)	WebSocket 위에서 pub/sub 의미를 표현하는 메시징 규약. `SUBSCRIBE /topic/...`으로 구독, `SEND /app/...`으로 발행
STOMP 브로커 켜는 스위치	`@EnableWebSocketMessageBroker`	Spring의 STOMP 메시지 브로커 기능을 활성화하는 설정 어노테이션. 엔드포인트·프리픽스 등록 시 `WebSocketMessageBrokerConfigurer`를 구현

컨트롤러로 들어오는 프리픽스	`setApplicationDestinationPrefixes("/app")`	클라이언트가 `/app/...`로 보낸 메시지를 서버 `@MessageMapping` 메서드로 라우팅하도록 하는 설정
구독 채널 프리픽스	`enableSimpleBroker("/topic")`	Spring 내장 인메모리 브로커를 활성화하며, `/topic/...`로 발행된 메시지를 해당 구독자에게 브로드캐스트
서버에서 메시지 밀어 주는 헬퍼	`SimpMessagingTemplate.convertAndSend(destination, payload)`	서버 코드에서 임의의 목적지로 STOMP 메시지를 발행할 때 쓰는 컴포넌트. 브로커를 거쳐 구독자에게 전달
서버 라우팅 규칙	`@MessageMapping("/chats")`	`/app/chats`로 들어온 STOMP 메시지를 처리할 메서드를 지정. 서버 쪽에서는 프리픽스(`/app`)를 생략하고 뒷부분만 기술
STOMP.js 클라이언트	`Stomp.over(new WebSocket(url))`	브라우저에서 WebSocket 위에 STOMP 프레임 처리를 얹는 JavaScript 라이브러리의 진입점. `connect`·`subscribe`·`send` API 제공

이것만은 기억하자

- **세 방식의 핵심은 "방향"과 "빈도"다.** 단방향인지 양방향인지, 사건이 가끔인지 지속인지. 이 두 축으로 Polling·SSE·WebSocket이 자연히 나뉜다
- **Polling의 비용은 "변화가 없어도 발생한다"는 데 있다.** 초당 요청 수는 **사용자 수 × 1/폴링주기**로 선형 증가한다. 유효 응답 비율은 대개 1% 미만
- **SSE는 "한 번 열어 두고 밀어 주는" 단방향 스트림이다.** `produces = MediaType.TEXT_EVENT_STREAM_VALUE`가 연결을 끊지 않게 하고, `SseEmitter.send(...)`가 이벤트를 밀어 준다. `ConcurrentHashMap` 레지스트리와 `onCompletion` / `onTimeout` 정리는 필수
- **WebSocket(STOMP)은 "목적지"로 말한다.** `/app/**`으로 SEND하면 서버 `@MessageMapping`이 받고, `/topic/**`으로 발행하면 구독자 전원에게 브로드캐스트된다. 프리픽스 설정이 라우팅 지도 전체를 만든다
- **scale-out이 다음 문제다.** 한 서버일 때는 세 방식 다 `SseEmitter`나 Simple Broker의 인메모리 맵으로 충분하다. 사용자가 늘고 서버가 여러 대가 되면, 연결이 나뉘면서 "이 이벤트를 다른 서버의 구독자에게 어떻게

전달하나"가 새 문제로 떠오른다